

At the Faculty of Computer and Information Science, students face difficulty selecting elective courses due to the large number of options. We propose a chat assistant that helps students choose courses based on their interests, ECTS requirements, and scheduling constraints. Such a chatbot would greatly benefit the FRI faculty by reducing the administrative burden on staff, who currently spend significant time answering individual student inquiries about course selection, while simultaneously helping students make more informed decisions about their academic paths.

Provisional dataset

The main source of information our chatbot will use are PDF files containing in-depth information about every course that the Faculty offers [4, 5]. These contain general information about each course, the number of credits they give, the semester and year they are held in, the field of study the course is meant for, as well as some other details. We will also try to obtain information about the time table for the current year [6]. We will also include all generally available data on the FRI website [7].

Methods

Document Processing and Indexing

The knowledge base for our chatbot is built from PDF course catalogues for the University of Ljubljana, Faculty of Computer and Information Science. These documents are first converted to Markdown format using the `pymupdf4llm` library, which preserves tabular structures and headings relevant to course descriptions. The resulting Markdown files are stored locally and serve as the primary input for the retrieval pipeline.

For indexing, documents are split into overlapping chunks using LangChain's

`RecursiveCharacterTextSplitter` with a chunk size of 1000 characters and an overlap of 150 characters. Each chunk is then embedded using the `sentence-transformers/all-MiniLM-L6-v2` model, which produces L2-normalised 384-dimensional dense vectors. The resulting embeddings are stored in a FAISS vector index that is persisted to disk and can be reloaded between sessions.

Retrieval

At query time, the user's question is first preprocessed by expanding domain-specific abbreviations (e.g., *uni* → *univerzitetni program*, *ects* → *kreditne točke*). The processed query is then embedded with the same model used during indexing and used to retrieve the k most relevant chunks from the FAISS index. We support two retrieval strategies: standard cosine similarity search and Maximal Marginal Relevance (MMR), which balances relevance and diversity among retrieved chunks.

The value of k is determined dynamically. Before each query, a dedicated model call extracts any explicit quantity mentioned in the question (e.g., "give me 3 courses"); if a number is found, k is scaled accordingly. This allows the system to adapt the size of the retrieved context to the user's informational need.

Generation

We use Meta's `Llama-3-8B-Instruct` model as the generative backbone, loaded in 4-bit NormalFloat quantisation (NF4) with double quantisation enabled via `BitsAndBytes`, allowing inference on a single consumer GPU. The model is wrapped in a `LangChain HuggingFacePipeline` with

greedy decoding (`do_sample=False`) and a repetition penalty of 1.1 to reduce degenerate outputs.

Retrieved chunks are concatenated and injected into a structured system prompt that instructs the model to answer exclusively from the provided context and to always include the course name, ECTS credits, and semester when listing courses. The prompt is formatted using the model's built-in chat template via `apply_chat_template`.

Conversational Memory

The system supports multi-turn dialogue by maintaining a conversation history as a list of question-answer pairs. This history is serialised to a JSON file after each turn and reloaded at the start of the next session, ensuring continuity across separate invocations of the pipeline. The full history is prepended to each new prompt so the model can resolve references to previous turns.

RAG Pipeline

The complete pipeline is implemented using LangChain's `LCEL` (LangChain Expression Language) and consists of four stages: (1) context retrieval, (2) prompt construction, (3) language model generation, and (4) output parsing. The pipeline accepts a dictionary containing the current question and conversation history, and returns the model's answer as a plain string.

Results

To evaluate the performance of the proposed RAG-based chatbot, we conducted a series of test queries covering different levels of complexity. The queries ranged from simple factual lookups to more detailed, multi-course descriptions. Below we present and analyse the results of four representative test cases.

Simple Factual Queries

For straightforward queries, the system performed reliably. When asked to name two courses related to embedded systems, the chatbot correctly identified *Vgrajeni sistemi* (Embedded Systems) and a second course involving PIC microcontrollers. Similarly, when asked about courses covering machine learning and artificial intelligence, the system returned a list of eight relevant courses, including *Strojno učenje*, *Umetna inteligencija*, *Računalniška vidnost*, and *Spodbujevalno učenje*. While all of these courses exist, the system does make up the names of some courses (eg. "Računalniška vidnost", with the real name of the course being "Umetno zaznavanje").

These results demonstrate that for well-scoped queries with a clear answer present in the retrieved context, the pipeline retrieves appropriate chunks and the language model produces accurate, grounded responses.

Multi-Course Detail Queries

When the user requested more detail about two specific courses — *Deep Learning* and *Computer Vision* — the system pro-

duced structured descriptions for both, including topics covered and learning goals. While the high-level structure of the responses was coherent and the course names were correctly identified, we observed the first signs of hallucination at this level of complexity. Specifically, the course *Gladko učenje* was listed as the Slovenian title for Deep Learning, which is incorrect; the accurate translation is *Globoko učenje*. Additionally, some of the listed subtopics (e.g., specific frameworks and techniques) could not be verified against the source documents, suggesting the model partially generated content beyond what the retrieved context contained.

Complex Thematic Queries

For more open-ended queries, such as asking for all courses related to software architectures and requesting detailed descriptions of each, the quality of responses degraded noticeably. While the system did identify two plausible courses — *Architecture Design and Implementation and Advanced Topics in Software Engineering* — the detailed content attributed to these courses included topics that do not appear in the source PDFs, such as specific architectural patterns (MVC, EDA, pipe and filter) and methodological frameworks (Kruchten's 4+1), which were either absent from or only briefly mentioned in the retrieved chunks.

This behaviour is consistent with a known failure mode of RAG systems: when the retrieved context is insufficient to fully answer a complex query, the language model falls back on its parametric knowledge and generates plausible-sounding but unverified information. In a practical deployment, this is particularly problematic because the fabricated course details could mislead students during course selection.

Summary

Overall, the system shows promising performance for simple retrieval tasks, but struggles with complex queries that require synthesising information across many documents or generating detailed descriptions not explicitly present in the source material.

Discussion

Use the Discussion section to objectively evaluate your work, do not just put praise on everything you did, be critical and exposes flaws and weaknesses of your solution. You can also explain what you would do differently if you would be able to start again and what upgrades could be done on the project in the future.

Acknowledgments

Here you can thank other persons (advisors, colleagues ...) that contributed to the successful completion of your project.

References

- [1] Bhavika R. Ranoliya, Nidhi Raghuvanshi, and Sanjay Singh. Chatbot for university related faqs. In *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 1525–1530. IEEE, 2017.
- [2] Fabio Clarizia, Francesco Colace, Marco Lombardi, Francesco Pascale, and Domenico Santaniello. Chatbot: An education support system for student. In *Cyberspace Safety and Security: 10th International Symposium, CSS 2018*, volume 11161 of *Lecture Notes in Computer Science*, pages 291–302, Cham, 2018. Springer.
- [3] Jakub Swacha and Michał Gracel. Retrieval-augmented generation (rag) chatbots for education: A survey of applications. *Applied Sciences*, 15(8):4234, 2025.
- [4] University of Ljubljana, Faculty of Computer and Information Science. Računalništvo in informatika (uni). https://www.fri.uni-lj.si/upload/Zborniki/1000468_Ra%C4%8Dunalni%C5%A1tvo_in_informa%20-%20Copy%2014.pdf, n.d. Accessed: 2026-03-19.
- [5] University of Ljubljana, Faculty of Computer and Information Science. Računalništvo in informatika (vss). https://www.fri.uni-lj.si/upload/Zborniki/1000470_Ra%C4%8Dunalni%C5%A1tvo_in_informa%20-%20Copy%2013.pdf, n.d. Accessed: 2026-03-19.
- [6] University of Ljubljana, Faculty of Computer and Information Science. Urnik fri. https://urnik.fri.uni-lj.si/timetable/fri-2025_2026-letni/, n.d. Accessed: 2026-03-19.
- [7] University of Ljubljana, Faculty of Computer and Information Science. Ul fri. <https://www.fri.uni-lj.si/sl>, n.d. Accessed: 2026-03-19.